



MIDTRANS SNAP API DOCUMENTATION

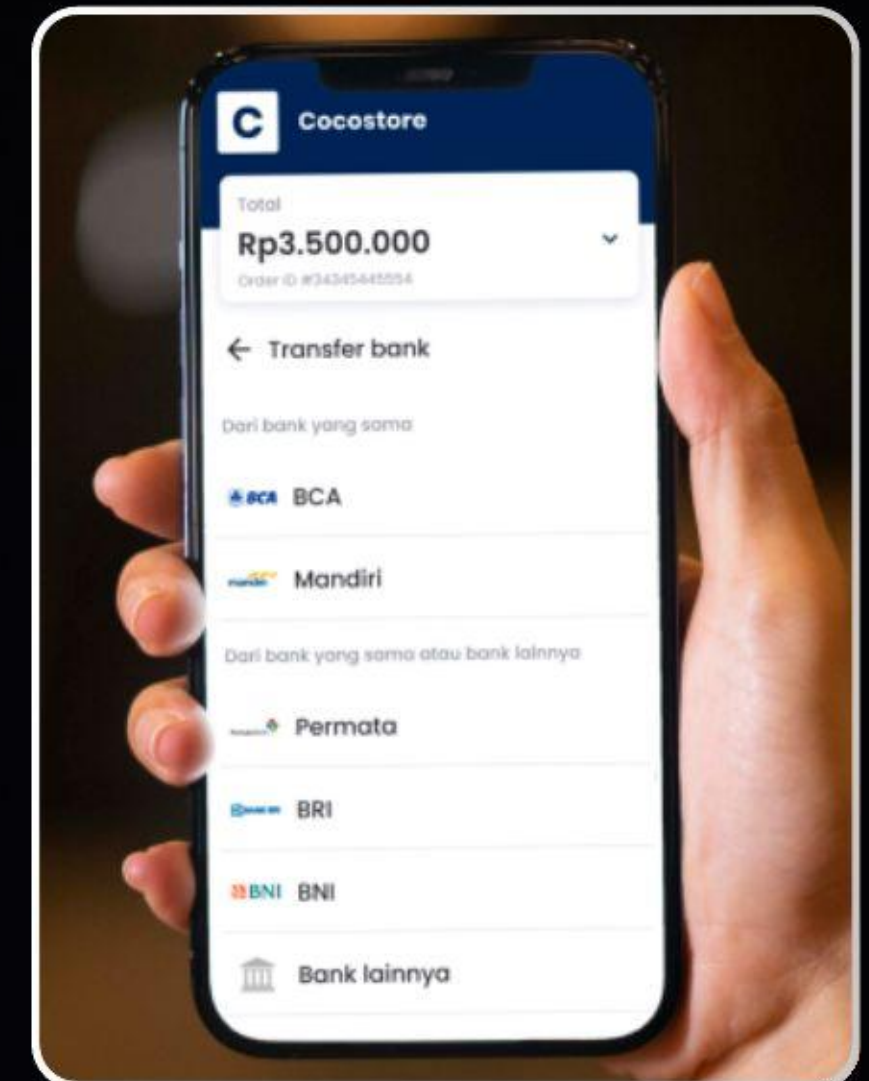
Via Built-in Interface Snap

GROUP 2 – ABID ARROFI | SAYYIDATUNISA | JENNA GOZALI

Overview

Snap is Midtrans's built-in payment interface, enabling merchants to accept payments through a single integration.

- Hosted payment page: Popup, Embedded, or Redirect
- Broad payment support: bank transfer/VA, cards, e-wallet, QRIS, and paylater
- No monthly fees
- Ideal for startups to enterprises with more specific UX requirements





Value Proposition & Key Advantages

01



Comprehensive Payment Method Access

With a single integration, you get direct connectivity to Virtual Accounts, Credit/Debit Cards, E-Wallets, QRIS, and Pay-later.

02



No Monthly Fees

Free of subscription fees, making it ideal for startups through to large-scale businesses.

03



Low PCI-DSS Burden

Because the payment page is hosted by Midtrans, credit card transaction data is better secured and the platform's compliance burden is reduced.

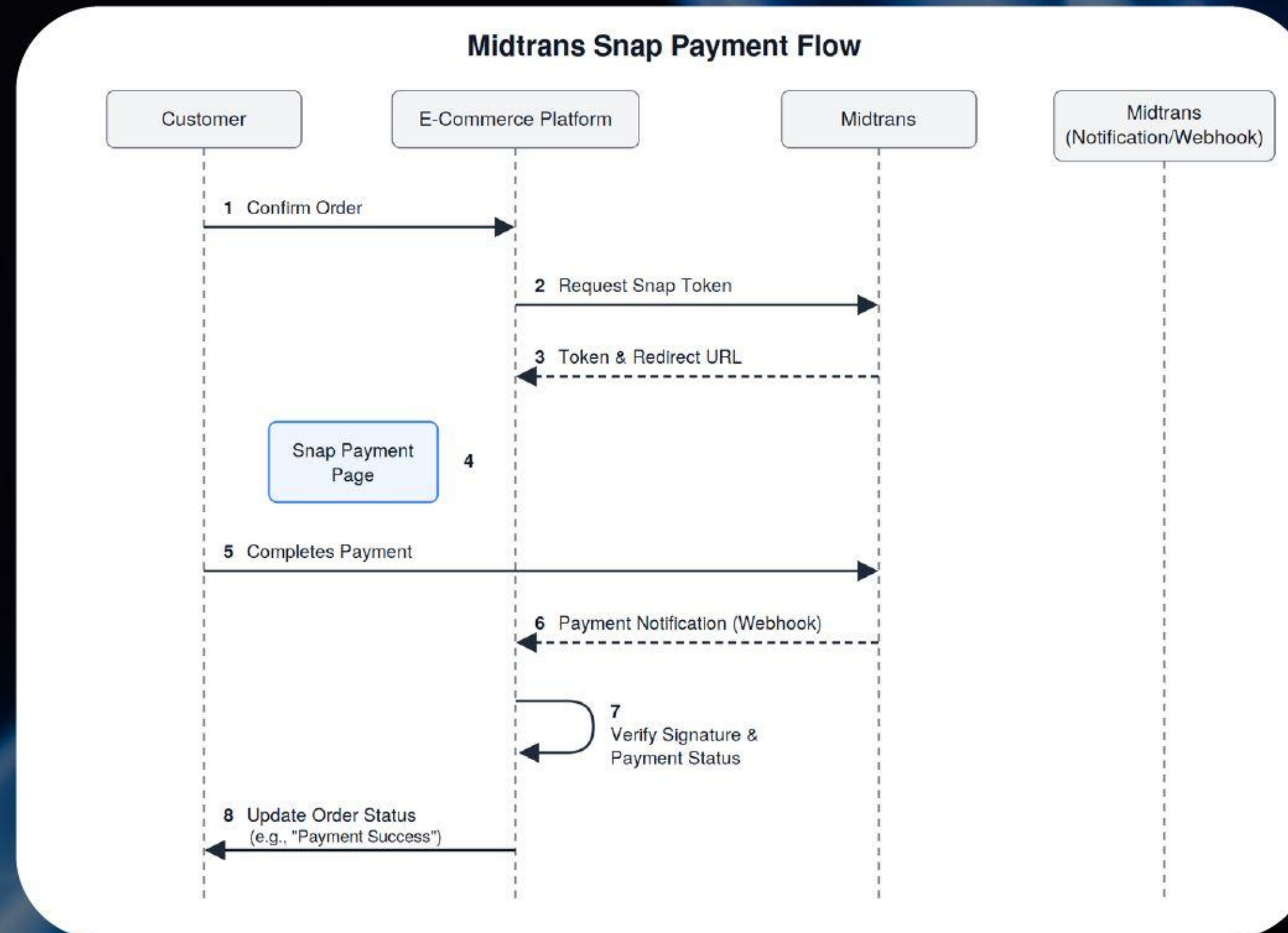


Three Snap Display Modes

- **Snap Popup Mode:** Displays the payment page as an overlay on top of the merchant's website via the *window.snap.pay(token)* function.
- **Snap Embedded Mode:** Embedded directly within a `<div>` element on the merchant's checkout page via *snap.embed(token, { embedId })*.
- **Snap Redirect Mode:** Redirects the buyer to the Midtrans payment page via *redirect_url* (suitable for mobile WebView checkout flows).



Sequence Diagram





Payment Notification (Webhook) Handling

- **HTTP POST Notification:** Sent automatically by Midtrans to the merchant's backend endpoint whenever a transaction status changes.
- **Signature Security Verification:** You must recompute the *signature_key* using SHA512 (*order_id* + *status_code* + *gross_amount* + *ServerKey*) before processing the order.
- **Idempotency Principle:** The webhook handler must be designed to withstand repeated notifications for the same transaction.

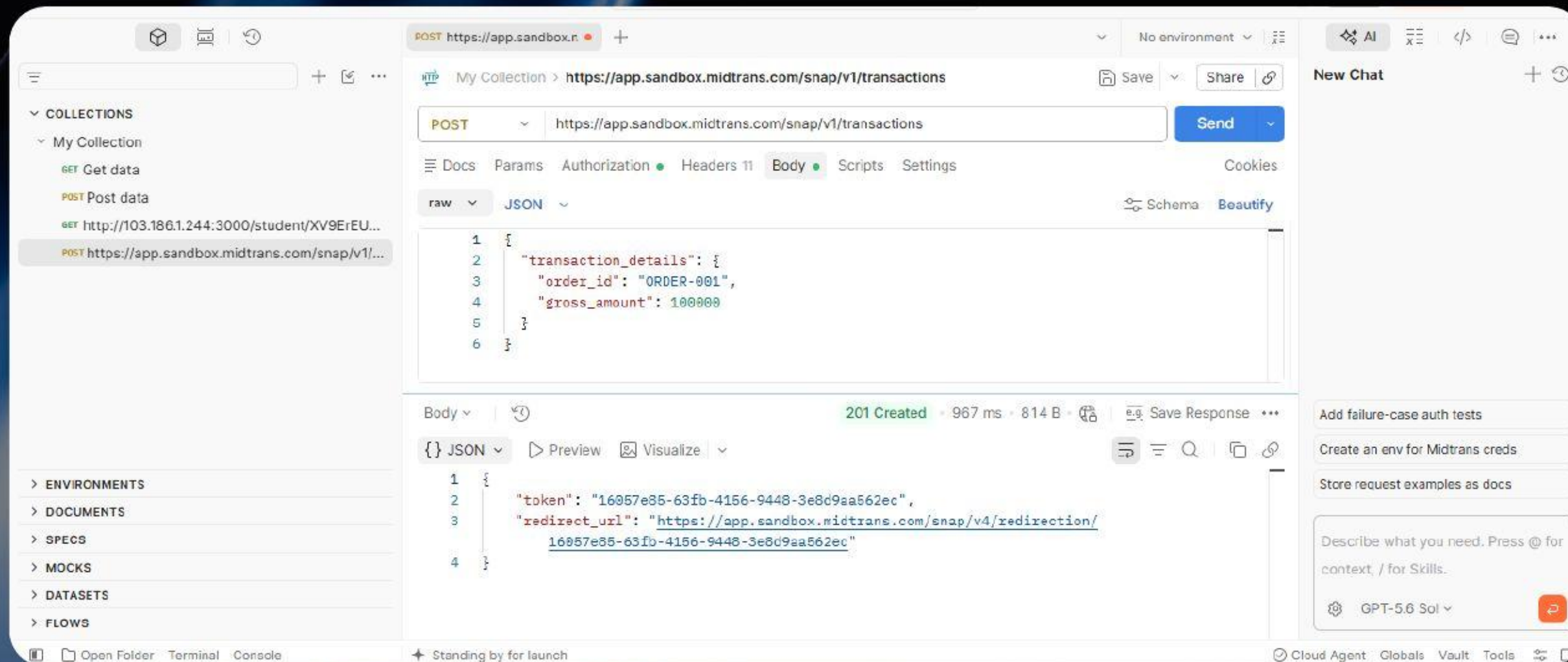


Key Use Cases

- **Automated Checkout Payment:** Buyers pay directly on the platform without having to leave the checkout flow.
- **Real-Time Order Confirmation:** Order status automatically changes to "Paid" as soon as the payment settles via webhook.
- **WebView Mobile App Integration:** Uses Redirect/Embedded mode inside a mobile app's WebView, with no separate SDK required.
- **Fallback Verification:** Uses the Get Status API if there is a delay in receiving the webhook.



API Request & Create Snap Transaction



API Request

- **Method:** POST
- **Endpoint Sandbox:** [POST https://app.sandbox.midtrans.com/snap/v1/transactions](https://app.sandbox.midtrans.com/snap/v1/transactions)
- **Authentication:** Basic Auth using the Server Key
- **Authentication Header:** `Authorization: Basic [Base64(ServerKey)]`.
- **Payload:** Unique order_id + gross_amount



Token Field & API Response

```
Body ▾ | ↺ | 201 Created • 967 ms • 814 B • 🌐 | e.g. Save  
{} JSON ▾ | ▶ Preview | 🖼 Visualize ▾ | ⌵ ⌵  
1 {  
2   "token": "16057e85-63fb-4156-9448-3e8d9aa562ec",  
3   "redirect_url": "https://app.sandbox.midtrans.com/snap/v4/redirection/  
4     16057e85-63fb-4156-9448-3e8d9aa562ec"  
5 }  
★ Standing by for launch
```

Token

The transaction token used to open the Snap payment page.

Redirect_url

The URL returned by Midtrans to send the customer to the payment page.

API Request → 201 Created → Snap Token + Redirect URL → Payment Page



Step-by-Step Implementation

1. **Onboarding:** Register a Midtrans Merchant Administration Portal (MAP) account and obtain the Sandbox API Keys.
2. **Development:** Build the token request on the backend and install the *snap.js* script on the frontend.
3. **Testing:** Simulate successful, pending, and error transaction scenarios in the Sandbox environment.
4. **Integration & Mapping:** Configure Snap Preferences and map to internal order statuses.
5. **Production Deployment:** Submit business verification, switch to Production API Keys, then go live.



Resource Allocation (Resource Requirements)

Technical Team

- **Backend Developer:** Integrates the token request API and webhook signature verification.
- **Frontend Developer:** Integrates snap.js and handles UI callbacks.
- **QA Engineer:** Validates Sandbox test scenarios.

Non-Technical Team

- Product Manager
- Business Analyst (status mapping)
- Customer Support
- Compliance/Legal



Integration Timeline Estimate

- **Preparation Phase:** MAP registration & Sandbox credentials (2–3 days).
- **Development Phase:** Backend API & frontend snap.js integration (1–2 weeks).
- **Testing & Configuration Phase:** Sandbox testing & Snap Preference setup (3–5 days).
- **Production Verification Phase:** Business document submission & key replacement (3–5 days).
- **Go-Live & Monitoring:** Deployment (1–2 days) & ongoing monitoring.



Pros & Cons Analysis

Aspect	Pros	Cons
Reliability	Webhooks deliver transaction status updates to the backend.	Heavily dependent on the reliability of the merchant's backend webhook system.
Scalability	Covers all payment channels in a single integration.	Adding specific payment/UX requirements may still require additional configuration.
Ease of Implementation	Simpler implementation; reduces frontend development effort and PCI-DSS audit scope.	UI customization is more limited compared to using the Core API.
Security	Includes mechanisms to protect credentials and validate notifications.	Security still depends on the merchant's implementation.



Practical & Security Considerations

Security

- **Challenge:** Server Key exposure creates a risk of unauthorized access.
- **Mitigation:** Store the Server Key on the backend only.

Webhook

- **Challenge:** Notifications may fail or be processed late.
- **Mitigation:** Use an idempotent handler + logging.

Transaction Status

- **Challenge:** Order status can become inconsistent if you rely on a single parameter.
- **Mitigation:** Use transaction_status + fraud_status as the reference.

Monitoring

- **Challenge:** API/webhook errors make troubleshooting difficult.
- **Mitigation:** Store request/response logs and errors.



Advanced Features

Recommended Parameters & Callbacks

- **Recommended Parameters:** It is advisable to include *customer_details* and *item_details* (including billing/shipping address) to provide complete data context.
- **Custom Finish URL:** Overrides the dashboard's default redirect URL for specific transactions via the *callbacks.finish* parameter when creating the token.

Snap.js Controls & Payment Filters

- **Snap.js Functions & Callbacks:**
 - *window.snap.pay* & *window.snap.hide*.
 - Handler callback: *onSuccess*, *onPending*, *onError*, and *onClose*.
- **Channel Restriction:** Send the *enabled_payments* array in the request payload to display only specific payment methods (e.g., [*"credit_card"*, *"gopay"*, *"bca_va"*]).



Expiry, Metadata & Fee Imposition

- **Transaction Expiry Time:** Sets the payment time limit per payment method (*expiry*) as well as how long the Snap page stays active (*page_expiry*).
- **Custom Fields:** Uses *custom_field1-3* to display custom text or internal order metadata.
- **Automatic Fee Imposition:** Automatically adds the MDR to the buyer via *customer_imposed_payment_fee*.

Credit Cards & Special Features

- **3D Secure (3DS):** Enables *credit_card.secure: true* to reduce fraud risk.
- **Save Card & Subscription:** Stores card data via *save_card* & *user_id* for one-click checkout or recurring transactions (*saved_token_id*).
- **Routing & Installment:** Configures BIN filters (*whitelist_bins/blacklist_bins*), bank routing, and online/offline installments.



Conclusion & Recommendations

Conclusion

Midtrans Snap enables e-commerce businesses to offer multiple payment methods through a single integration with lower frontend effort.

Recommendations

I Secure the Integration

Store the Server Key on the backend and perform signature verification on webhooks.

I Ensure Payment Reliability

Use webhooks with an idempotent handler and provide the Get Status API as a fallback.

I Test Before Go-Live

Validate success, pending, and error scenarios in the Sandbox before switching to Production API Keys.

I Choose the Right Display Mode

Choose Popup, Embedded, or Redirect based on your UX and platform requirements.



Thank You

For Your Attention